AskQL

Governed answers, in plain English. Novelties, sources and future scope.

AskQL is an agentic business-analytics chatbot. A user asks a question in plain English. The system generates SQL against a governed metric layer, runs it, picks a chart, renders it and narrates one sentence of insight. It runs on **Northbeam**, a fully simulated B2B SaaS company (24 products, 5 categories). All data is synthetic.

> **Honest framing.** Text-to-SQL is a crowded category and the chatbot itself is not the novelty. The contribution is the set of design choices below, which make answers *trustworthy and fast* instead of merely plausible. Most individual ideas have precedent (see Sources). The novelty is in how they are combined and enforced.

1. Novelties

1.1 Governed answers: the agent never sees the raw schema

The LLM reads only YAML metric definitions (validated by Pydantic, failing loudly on a malformed one). If a question does not map to a defined metric, the correct behaviour is to say so. The agent never improvises SQL against raw tables, so "revenue" means one thing, defined once as a business decision.

1.2 Ambiguity asked, not guessed, and remembered

When two or more metrics plausibly match, the system asks a clarifying question (rendered as clickable chips) instead of guessing. The answer is resolved to a specific metric *in code* via keyword-overlap matching, not by trusting a small LLM to re-correlate its own prior question, and is remembered for the whole session.

1.3 Deterministic wherever a correct answer exists

Anything with a verifiable answer is computed in code, not delegated to the LLM:

- **Chart type** is a pure function of result shape (or an explicit request like "as a pie chart"). The LLM never generates images; it only emits `{chart_type, x, y, series}` JSON and Recharts renders it client-side.
- **Clarification resolution** is a keyword match, as above.
- **Narration facts**: highest and lowest values are pre-computed over the full result and handed to the LLM as a verified fact block. This fixed a real hallucination where the narration named a product 4.5x too small as "highest".

1.4 Layered SQL guardrails in place of database permissions

DuckDB has no role-based permissions, so safety is built in layers: `sqlglot` parses every statement and requires exactly one `SELECT` over the five governed tables (writes, DDL, table functions such as `read_csv`, file paths and other schemas are rejected); execution uses a read-only connection with external access disabled; and SQL pinned to a dashboard is **re-validated on every run**, since stored SQL is untrusted input again once read back.

1.5 Trust mechanics as a product feature

Every response carries the SQL used and the metric definition applied. The agent honestly refuses out-of-scope questions ("what's the weather") and retries a failed query only once before giving up honestly, rather than looping.

1.6 Fully reproducible simulated company

Every fact comes from a seeded formula: `base × (1+growth)^day × weekday_factor × seeded_noise × seeded_anomaly`, seeded per `(entity, product, day, row)` via `random.Random(str)`. No bare `random()` anywhere, so regenerating history reproduces it exactly and a seeded anomaly day is a specific event the bot can explain. Fact tables never duplicate dimensions; region and channel are joined through `customers` at query time, like a human analyst's lookup.

1.7 Conversational memory that survives restarts

Continuations ("now the same for 7 days") keep the same metric and change only what is explicit. Chart-type follow-ups ("now as a table") reuse the prior turn's rows with no new LLM call or query, so the SQL cannot drift. Context is checkpointed in SQLite, verified across an API restart.

1.8 Live, not snapshot, dashboards

A pinned item stores only the question, SQL, chart shape and narration, never data. Every load re-runs the SQL against the live database. Dashboard and chat metadata live in separate SQLite files so user-driven writes never contend with DuckDB's single writer.

1.9 Power BI-style report canvas

Dashboards are multi-page, free-form canvases with drag, 8-handle resize, smart-guide snapping, per-visual formatting and a rich text editor. Stored rich text only ever passes an allowlist HTML sanitizer. Charts render at real screen size (not CSS-transformed) so hover maths stays exact.

1.10 Engineering process

Three test tiers close three different blind spots: `pytest` for wiring and validation; a golden eval (`agent/eval.py`, about 20 scenarios against the *real* LLM and database) for SQL correctness; Playwright e2e for rendering and interaction bugs. Every bug was root-caused by measurement (DOM bounding boxes, computed styles), not guessed.

1.11 Latency as a design constraint

Target p95 under one second. This drove the stack: embedded DuckDB over a warehouse (cold starts of 30s or more recreate the waiting problem), deterministic chart and narration logic, and no extra LLM calls on follow-ups. *Note: the target is not yet measured; see Future Scope.*

2. Sources

The ideas and tools AskQL builds on or is inspired by. Prior art is credited here, not claimed as novel.

2.1 Conceptual prior art

Idea	Prior art	How AskQL uses it

Semantic / metrics layer	Looker (LookML), dbt Semantic Layer / MetricFlow	Metrics defined once as code in YAML; the agent reads these instead of raw columns
Text-to-SQL	Commercial BI copilots; academic benchmarks such as Spider and BIRD	Baseline category; AskQL differs by governance and ambiguity handling
Bronze/gold data split	Medallion architecture	Parquet raw archive plus DuckDB, without a warehouse
Dashboard canvas	Power BI report canvas	Pages, free placement, focus mode, format pane
Allowlist sanitising	Standard web security practice (XSS prevention)	Stdlib sanitizer for stored rich text

2.2 Technology stack

Layer	Tool
Storage	DuckDB (single file), Parquet; SQLite for chats, dashboards and checkpoints
Semantic layer	YAML, Pydantic
Agent	LangGraph (with SqliteSaver checkpointing)
SQL validation	sqlglot
LLMs	Google Gemini (primary), Ollama with `qwen2.5-coder:7b` (local fallback)
Backend	FastAPI, Server-Sent Events
Frontend	Next.js, React, TypeScript, Tailwind CSS v4, Recharts
Testing	pytest, Playwright
Design	Geist typeface; CVD-safe, contrast-checked chart palette from the `dataviz` skill

2.3 Data sources

- **Current:** none external. All data comes from the project's own deterministic simulator (`ingestion/company.py` and `ingestion/simulate.py`). Region and channel splits, tier weights and ticket rates are hand-set, tunable constants, not measured values.
- **Parked (unused):** real-API ingestion for GitHub, npm, PyPI and Hacker News (`ingestion/sources/`). Abandoned due to rate limits, a platform-side block on GitHub's stargazers/subscribers endpoints, and having only one real "product".

2.4 Documents in the repo

`CLAUDE.md` (working brief), `docs/askql-project-spec.md` (design rationale), `docs/phase-8-plan.md`, `README.md`.

3. Future Scope

3.1 Planned (roadmap)

Item	Why it matters
Latency measurement against p95 < 1s	The product thesis is currently asserted, not measured. Any regression should be flagged.
Query timeouts and row caps at execution time	Guardrail gap: validation limits *what* runs, not *how long* or *how large*.
Result caching	Single process, so `functools.lru_cache` suffices; no Redis needed.
Langfuse tracing	Observability for LLM calls, retries and per-node latency.
Proactive anomaly alerts	Seeded anomaly days already exist to detect. Undecided whether v1 or v2.
Authentication	Required before any public deploy.
Metric retrieval with DuckDB VSS	Part of the original design; needed once the metric catalogue outgrows fitting in a prompt.

3.2 Research and product directions

- **Real-data integration** needs a cleaning layer between ingest and fact tables: deduplication (the same sale synced twice from a CRM), null/type coercion, fuzzy ID resolution (a customer spelled three ways) and schema-drift handling. Deliberately not designed until a real source exists.
- **Deployment:** Docker to Railway/Render with a persistent volume, frontend on Vercel. Revisit when something needs to consume the API continuously.
- **Scaling past one instance** means moving from DuckDB to Postgres, trading latency for concurrency.
- **Broader metric coverage** and a wider golden eval set, as eval scenarios currently number about 20.
- **Rename before public release:** "AskQL" collides with an existing, unrelated project on npm and as a GitHub org.

4. Known Tradeoffs

- Single-writer DuckDB caps deployment at one API instance (deliberate, for latency).
- No database-level permissions; validation plus a read-only, no-external-access connection substitute for a read-only role.
- All company data is simulated, deterministic and internally consistent, not real.
- Text-to-SQL is crowded; the differentiation is governance and ambiguity handling.

Source links

Every link below is clickable. Names in Section 2 are linked too.

2.1 Conceptual prior art

Looker (LookML)	https://cloud.google.com/looker/docs/what-is-lookml
dbt Semantic Layer	https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl
dbt MetricFlow	https://docs.getdbt.com/docs/build/about-metricflow
Spider benchmark	https://yale-lily.github.io/spider
BIRD benchmark	https://bird-bench.github.io/
Medallion architecture	https://www.databricks.com/glossary/medallion-architecture
Power BI report canvas	https://learn.microsoft.com/en-us/power-bi/create-reports/
XSS prevention (OWASP)	https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

2.2 Technology stack

DuckDB	https://duckdb.org/
Parquet	https://parquet.apache.org/
SQLite	https://www.sqlite.org/
YAML	https://yaml.org/
Pydantic	https://docs.pydantic.dev/
LangGraph	https://langchain-ai.github.io/langgraph/
sqlglot	https://github.com/tobymao/sqlglot
Google Gemini API	https://ai.google.dev/gemini-api/docs
Ollama	https://ollama.com/
qwen2.5-coder	https://ollama.com/library/qwen2.5-coder
FastAPI	https://fastapi.tiangolo.com/
Server-Sent Events	https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events
Next.js	https://nextjs.org/
React	https://react.dev/
TypeScript	https://www.typescriptlang.org/
Tailwind CSS	https://tailwindcss.com/
Recharts	https://recharts.org/
pytest	https://docs.pytest.org/
Playwright	https://playwright.dev/
Geist typeface	https://vercel.com/font

2.3 Data sources (parked ingestion)

GitHub REST API	https://docs.github.com/en/rest
npm	https://docs.npmjs.com/
PyPI	https://pypi.org/
Hacker News API	https://github.com/HackerNews/API

3. Future scope and deployment

DuckDB VSS extension	https://duckdb.org/docs/extensions/vss
Langfuse	https://langfuse.com/
PostgreSQL	https://www.postgresql.org/
Railway	https://railway.com/
Render	https://render.com/
Vercel	https://vercel.com/
Docker	https://www.docker.com/

Sources by novelty

Prior art and tools behind each novelty in Section 1. Clickable.

1.1 Governed answers

Semantic/metrics layer: Looker LookML	https://cloud.google.com/looker/docs/what-is-lookml
Semantic/metrics layer: dbt metrics	https://docs.getdbt.com/docs/build/metrics-overview
Pydantic validation	https://docs.pydantic.dev/latest/
YAML	https://yaml.org/

1.2 Ambiguity asked, not guessed

Checkpointed session memory (LangGraph)	https://langchain-ai.github.io/langgraph/concepts/persistence/

1.3 Deterministic where possible

Recharts (client-side rendering)	https://recharts.org/
Text-to-SQL baselines: Spider	https://yale-lily.github.io/spider
Text-to-SQL baselines: BIRD	https://bird-bench.github.io/

1.4 Layered SQL guardrails

sqlglot parser	https://github.com/tobymao/sqlglot#readme
Securing DuckDB (read-only, no external access)	https://duckdb.org/docs/stable/operations_manual/securing_duckdb/overview

1.5 Trust mechanics

LangGraph graph and retries	https://langchain-ai.github.io/langgraph/concepts/persistence/

1.6 Reproducible simulation

Python random.Random	https://docs.python.org/3/library/random.html

1.7 Memory that survives restarts

LangGraph persistence / checkpointers	https://langchain-ai.github.io/langgraph/concepts/persistence/
SQLite	https://www.sqlite.org/whentouse.html

1.8 Live dashboards

DuckDB concurrency (one writer)	https://duckdb.org/docs/stable/connect/concurrency
SQLite locking	https://www.sqlite.org/lockingv3.html

1.9 Report canvas

Power BI reports	https://learn.microsoft.com/en-us/power-bi/create-reports/
XSS prevention (sanitizer)	https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

1.10 Engineering process

pytest	https://docs.pytest.org/en/stable/
Playwright	https://playwright.dev/docs/intro

1.11 Latency

Why DuckDB (embedded, in-process)	https://duckdb.org/why_duckdb
Bronze/gold split: Medallion architecture	https://www.databricks.com/glossary/medallion-architecture
Server-Sent Events streaming	https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events